

Experiment No. 4

Title: Practice TDD (Test-driven development) for the simple calculator application.

Problem description: Students are expected to write small part of the calculator program ex-adding functionality.

- ❖ **Student should run the code to test the add functionality using testing frameworks like Pytest, Junit test code parallelly.**
- ❖ **Next functionality is parallelly developed and tested.**

Aim:

To develop a simple calculator using the Test-Driven Development approach by writing test cases first and then implementing the functionality.

Software / Tools Required

1. Python / Java
2. Testing Framework: Pytest or JUnit
3. Jira for task management
4. Any IDE (VS Code / IntelliJ / Eclipse)

(Advice to write a python or java code for calculator function like add, subtract, multiply and divide only without any bugs – store it in your GitHub folder as Experiment Number 4)

About TDD (Test-Driven Development):

Jira Statis

- ❖ **To Do** - an use case/test case
- ❖ **In Progress** - of use case
- ❖ **Done** - the use case

TDD is simple: write a test code → see it fail → write code again → see it pass → repeat for another test.

TDD Cycle: Students must follow Red → Green → Refactor.

Red Means → Write Test First, fail it

Green Means → Pass It

Refactor Means → go to another use case

Step 1 – Create Project in JIRA

1. Login to JIRA.
2. Create a new project named Calculator-TDD.
3. Create the following issues:
 - ❖ User Story 1: Implement Addition.
 - ❖ User Story 2: Implement Subtraction.
 - ❖ User Story 3: Implement Multiplication.
 - ❖ User Story 4: Implement Division.

Step 2 – Create Development Task

1. Under User Story 1, create a task:
 - ❖ Write test case for Add function.
2. Move the task to To Do in the board.

Step 3 – Write Test First (RED Stage)

Example using Pytest

// python Code

```
from calculator import add
```

```
def test_add():
```

```
    assert add(2,3) == 5
```

Run the test.

// Expected Result: Test fails because the function is not implemented.

- ❖ This confirms the **RED stage**.
- ❖ Update JIRA task status as In **Progress**.

Step 4 – Write Minimal Code (GREEN Stage)

Create the calculator program.

// python Code

```
def add(a,b):
```

```
    return a+b
```

Run the test again.

//Expected Result: Test **passes**.

- ❖ This confirms the **GREEN stage**.
- ❖ Update JIRA task status as **Done**.

Step 5 – Refactor the Code

Improve code structure if needed.

Example:

```
def add(a:int,b:int)->int:
```

```
    return a+b
```

- ❖ Run tests again to ensure they still pass.

Step 6 – Repeat TDD for Next Function

Create new JIRA task:

Task: Implement Subtraction

Follow same cycle:

1. Write test case first
2. Run test → Fail
3. Write code
4. Run test → Pass

// python Code

```
def test_subtract():  
    assert subtract(5,3) == 2
```

Step 7 – Continue for Remaining Operations

Repeat the cycle for:

- ❖ Multiplication
- ❖ Division

Track each task in JIRA board workflow:

- ❖ To Do
- ❖ In Progress
- ❖ Done

Expected Output

1. Calculator program with four operations:
 - ❖ **Add**
 - ❖ **Subtract**
 - ❖ **Multiply**
 - ❖ **Divide**
2. Unit test cases executed successfully.
3. All tasks completed in JIRA board.

Advantages of TDD:

- ❖ Most beginners hate writing tests first (to overcome it)
- ❖ Encourages Incremental Development
- ❖ Integrates Well with Agile Development (Adopt changes)

Result

The simple calculator application was successfully developed using the TDD (Test-Driven Development) methodology, where test cases were written before implementing the program logic and verified using automated testing frameworks.